

Software Engineering

Lecture 6: Architectural Design

Prof. Nien-Lin Hsueh

Department of Information Engineering and Computer Science
Feng Chia University

Focus Questions

- What is **Architectural Design** and why is software architecture the critical link between requirements and design?
- How does **Architecture in the Small** differ from **Architecture in the Large**?
- What are Kruchten's **4+1 Architectural Views**?
- How do classic **Architectural Patterns** (MVC, Layered, Repository, Client-Server, Pipe & Filter, Microservices) structure software systems?
- How does architectural design impact system non-functional characteristics (Performance, Security, Safety, Availability, Maintainability)?
- How can **AI** assist software architects in Architectural Design, Pattern Selection, and Trade-off Analysis?

What is Architectural Design?

- **System Structure:**
 - Architectural design is concerned with understanding how a software system should be organized and designing its overall structure.
- **Critical Link:**
 - Connects **requirements engineering** to **detailed software design** by identifying principal structural components and their relationships.
- **Architectural Model:**
 - Outlines communicating components and system boundary interfaces.

[SYSTEM_CONTEXT] // DEFINING THE CHASM

Connecting the 'What' to the 'How'



Requirements Engineering (The What)

// Documenting customer needs & specifications

Software Design

Architecture:

Identifying principal structural components and their relationships.

// Defining high-level structure, modules, interfaces & interactions. Ensuring scalability, security, and performance.

Component Design:

Structuring internal algorithms, classes, and logic.

// Detailing low-level logic, data structures, functions, and specific algorithmic implementations.



Implementation (The Code)

// Generating executable software & algorithms

Architectural Abstraction Levels

- **Architecture in the Small:**
 - Concerned with individual programs or applications.
 - Focuses on how a single program is decomposed into sub-components, classes, and packages.
- **Architecture in the Large:**
 - Concerned with complex enterprise systems composed of interacting multi-system networks.
 - Systems are distributed across multiple servers, hardware cores, and cloud infrastructure managed by different teams.

Advantages of Explicit Architecture

1. Stakeholder Communication:

- High-level architectural diagrams act as a common focus of discussion between technical teams, business managers, and clients.

2. System Analysis:

- Enables early verification of whether the system can satisfy key **non-functional requirements** (e.g., performance, security).

3. Large-Scale Reuse:

- Core architectural styles can be reused across product lines in the same domain.

Architecture & System Characteristics

- **Performance:** Localize critical operations in large-grain components; minimize inter-component communication.
- **Security:** Use a **layered architecture** with critical data assets enclosed in inner layers.
- **Safety:** Isolate safety-critical features into dedicated, isolated sub-systems.
- **Availability:** Include redundant components and automated failover mechanisms.
- **Maintainability:** Use fine-grained, loosely coupled, replaceable components.

Architectural Views (Kruchten's 4+1 Model)

- **Logical View:** Shows key domain abstractions as object classes.
- **Process View:** Shows runtime interacting processes and threads.
- **Development View:** Shows code structure, packages, and sub-systems.
- **Physical View:** Shows hardware deployment and network distribution.
- **+1 Use Cases / Scenarios:** Connects all 4 views together.

[MACRO_LEVEL] // THE 4+1 VIEW MODEL

Five Perspectives, One Unified Blueprint

Logical View:
Object classes and key
domain abstractions.



Process View:
Runtime interacting
processes and threads.



Development View:
Code structure, packages,
and sub-systems.



Physical View:
Hardware deployment
and network distribution.



+1
Use Cases /
Scenarios

[SYS_NOTE: Solves Concept Check 1 -
Maps components to runtime hardware]

Concept Check Question 1

Which architectural view in Kruchten's 4+1 View Model illustrates how software components are deployed across physical hardware and network servers at runtime?

- **A.** Logical View
- **B.** Process View
- **C.** Development View
- **D.** Physical View



Concept Check Question 1: Answer

Which architectural view in Kruchten's 4+1 View Model illustrates how software components are deployed across physical hardware and network servers at runtime?

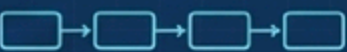
- **Correct Answer: D**
- **Explanation:** The Physical View (Deployment View) maps software components to physical hardware processors and network topology.



What is an Architectural Pattern?

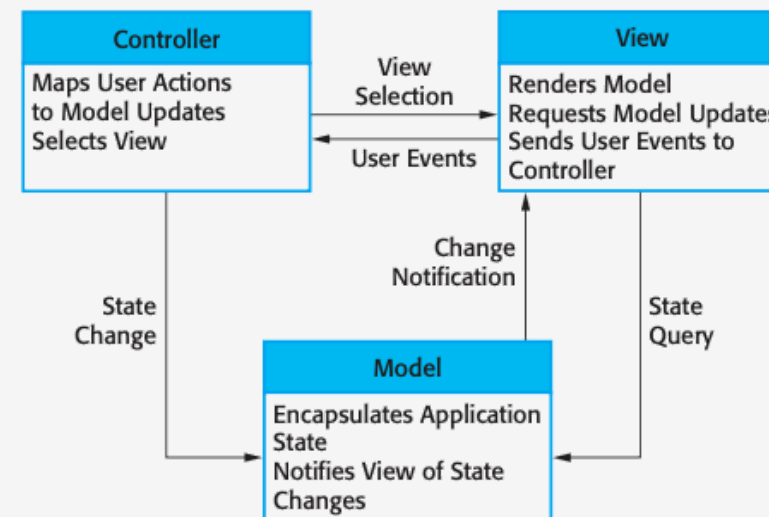
- **Reusable Architectural Knowledge:**
 - A stylized, tested description of good design practice applicable to common architectural scenarios.
- **Pattern Structure:**
 - Defines the pattern name, description, example usage, when to use, advantages, and disadvantages.
- **Common Patterns:**
 - **MVC, Layered Architecture, Repository Architecture, Client-Server, Pipe & Filter, Microservices.**

The Architectural Patterns Matrix

Pattern Name	Visual Icon (Data Flow)	Core Mechanism	Best Used For	Primary Risk/Trade-off
MVC	 [SYS_NOTE: Solves Concept Check 2 - Decouples presentation from data]	Separates UI/View from data/Model via Controller.	Interactive apps with multiple views	Overkill for simple data displays
Layered		Stacked abstract machines where each layer serves the one above.	Systems needing strict isolation	Performance overhead across layers
Repository		Sub-systems operate independently via shared central database state changes.	IDEs, Data-heavy applications	Single point of failure at database
Client-Server		Functionality split into specialized servers requested by clients over network.	Distributed scalable networks	Single point of failure per server
Pipe & Filter		Sequential discrete data transformations connected by streams.	Batch processing, Compilers	Bottlenecks in sequential processing

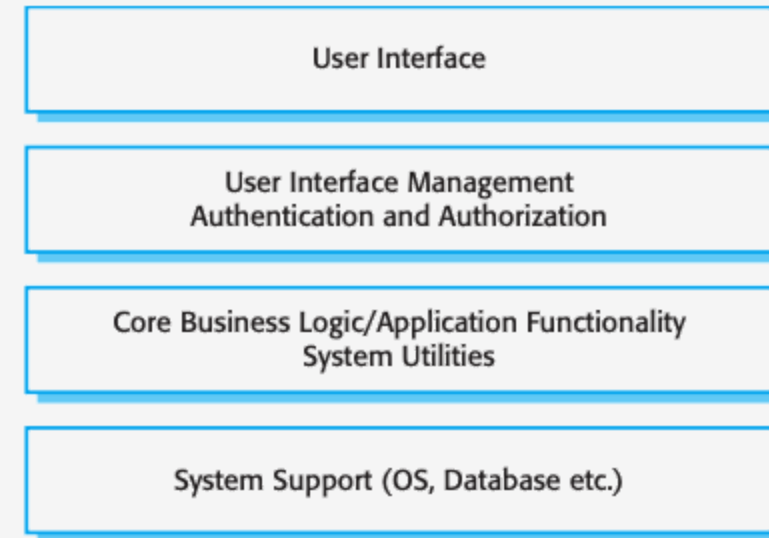
Model-View-Controller (MVC) Pattern

- **Model:** Manages system data and business logic.
- **View:** Manages presentation and UI layout to the user.
- **Controller:** Handles user input events and updates View/Model accordingly.
- **Advantage:** Separates data management from presentation, enabling multiple views of the same data.



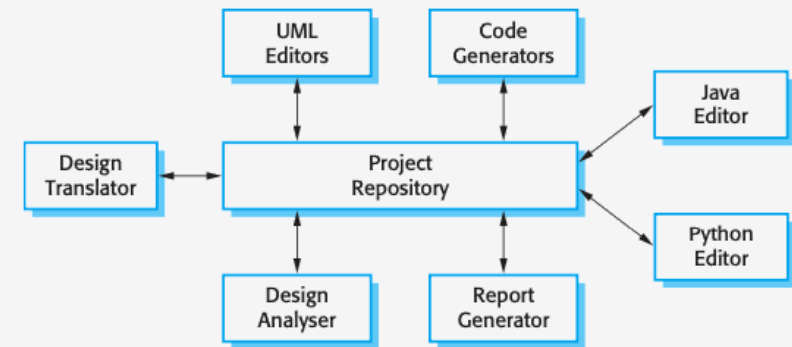
Layered Architecture Pattern

- **Layered Abstraction:**
 - Organizes system into stacked layers. Each layer provides services to the layer directly above it.
- **Core Layers:**
 - User Interface → User Interaction Management → Core Business Logic → System Database / OS.
- **Advantage:** Allows complete layer replacement if interfaces remain stable.



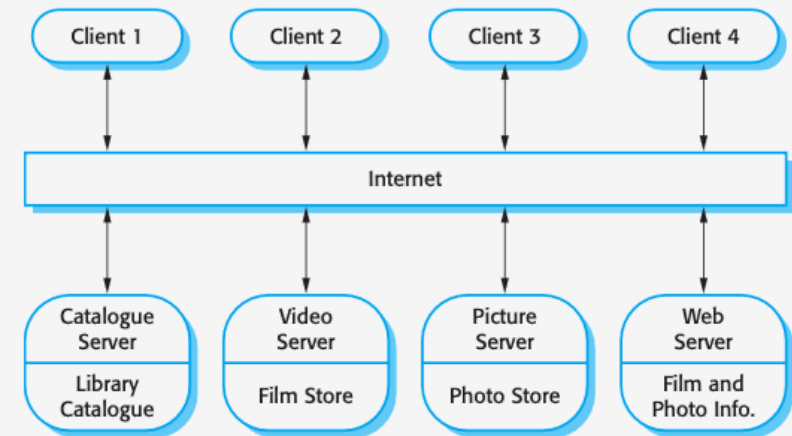
Repository Architecture Pattern

- **Centralized Data Sharing:**
 - Sub-systems exchange large volumes of data through a shared central database or repository.
- **Loose Component Coupling:**
 - Sub-systems operate independently; interactions occur via repository state changes.
- **Examples:** IDEs (Eclipse, VS Code), CAD systems, Medical Record databases.



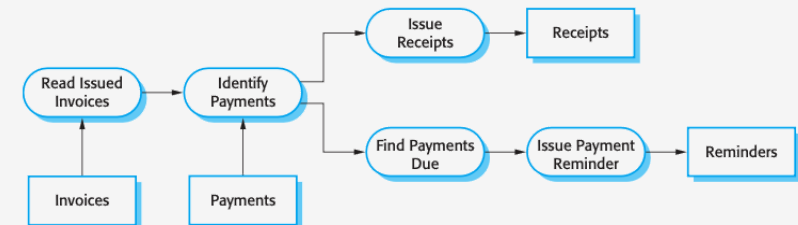
Client-Server Architecture Pattern

- **Distributed Services:**
 - Functionality is split into servers offering specialized services (database, printing, web) and clients making requests over a network.
- **Advantages:**
 - Servers can be distributed across different machines; scalable network infrastructure.
- **Disadvantages:**
 - Single point of failure per server; network latency dependencies.



Pipe and Filter Architecture Pattern

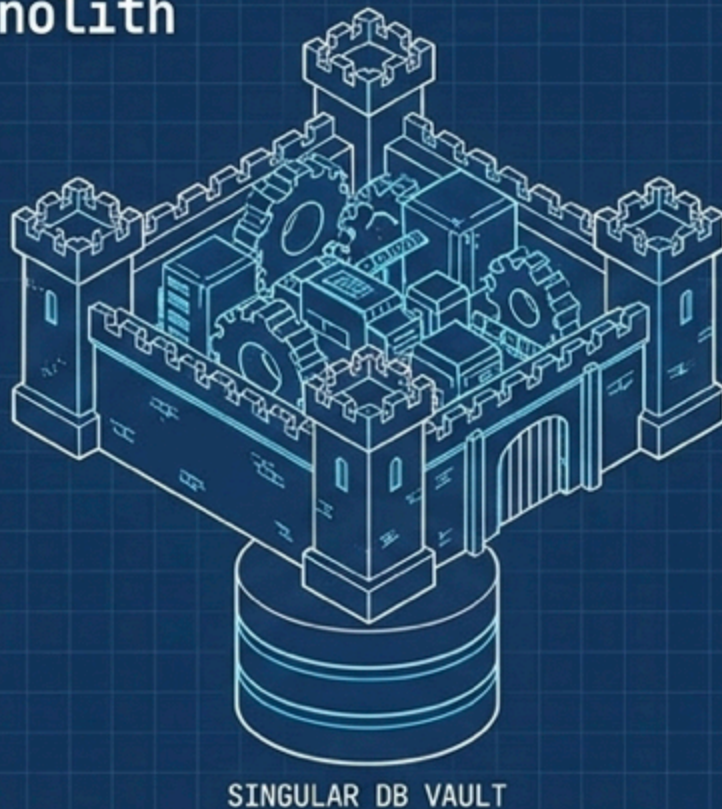
- **Sequential Data Transformation:**
 - Processing is structured as a series of discrete transformation steps (**filters**) connected by data streams (**pipes**).
- **Unix Shell Model:**
 - Output of one filter becomes the input stream of the next filter (e.g., `cat | grep | sort`).
- **Best Used For:** Batch data processing, invoice processing, compilers.



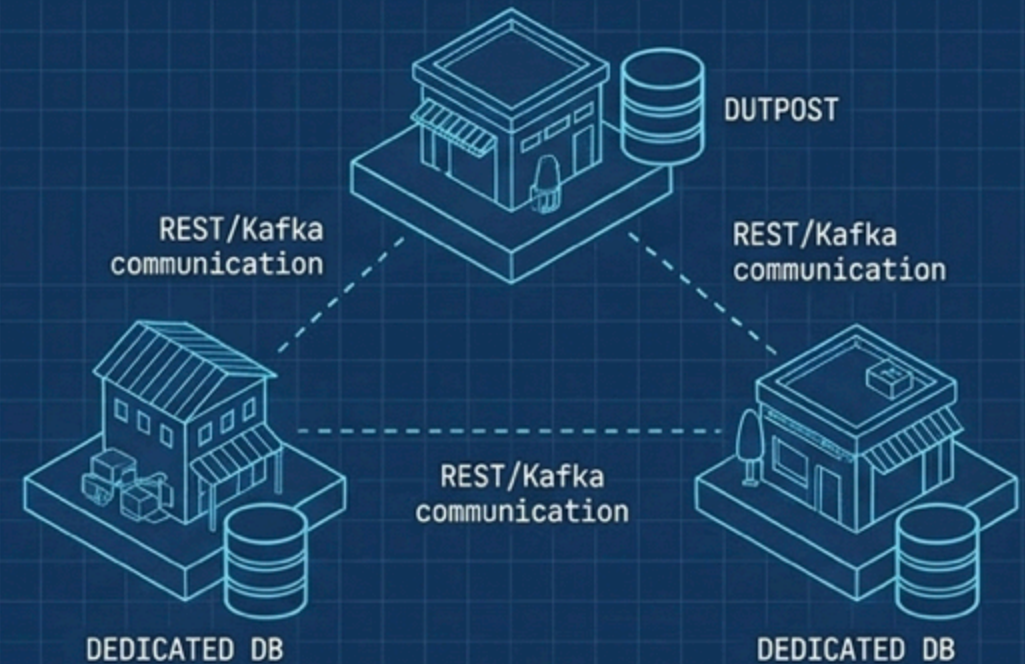
[MACRO_LEVEL] // THE SCALE SPLIT

Monolithic Fortresses vs. Microservice Networks

The Monolith



Microservices Architecture (MSA)



Microservices Architecture Characteristics

- **Decoupled Business Services:**
 - Application built as a suite of small, independent services (e.g., *User Service*, *Order Service*, *Payment Service*).
- **Independent Deployment & Databases:**
 - Each service manages its own database and can be updated, deployed, and scaled independently.
- **Lightweight Communication:**
 - Services communicate asynchronously or via lightweight REST APIs (HTTP / Kafka).
- **Resilience:**
 - Failure of one service does not crash the entire application.

AI in Architectural Design: Overview

- **The Architect Copilot Paradigm:**
 - LLMs assist software architects in structuring complex systems, selecting architectural patterns, and evaluating trade-offs.
 - Translates high-level non-functional requirements into structural blueprints.
- **Key Benefit:** Accelerates architecture trade-off evaluation and interface contract drafting.

The AI Architect Copilot Dashboard

Macro / Architecture Automation

- **Pattern Recommendation:** Analyzing non-functional requirements to suggest Microservices vs. Layered.
- **ATAM Analysis:** Evaluating conflicting attributes (Security vs. Latency).
- **Threat Modeling:** Scanning diagrams for Single Points of Failure (SPOF) and STRIDE threats.

Micro / Software Design Automation

- **SOLID Refactoring:** Identifying God Classes and suggesting factory patterns.
- **Code Smell Detection:** Highlighting tight coupling and duplicate logic.
- **Interface Generation:** Auto-generating OpenAPI specs, gRPC .proto files, and decoupled mocks for TDD.

4 Core AI Applications in Architectural Design

1. Architectural Pattern Recommendation:

- Analyzes non-functional requirements to recommend suitable architectural styles (Microservices, Event-Driven, Layered).

2. ATAM & Trade-off Analysis Assistance:

- Evaluates conflicting quality attributes (e.g., Security vs. Latency, CAP Theorem Consistency vs. Availability).

3. API & Interface Contract Generation:

- Generates OpenAPI (Swagger) specs, gRPC `.proto` schemas, and event payloads to decouple sub-systems.

4. Threat Modeling & Security Boundary Analysis:

- Scans system architecture diagrams for single points of failure (SPOF) and STRIDE security threats.

Human-in-the-Loop: Architectural Risks & Best Practices

- **Risks of Unchecked AI in Architecture:**

- **Architectural Blind Spots:** AI lacks context regarding team tech stack familiarity, cloud vendor costs, and organizational culture.
- **Premature Microservices:** AI recommending complex distributed architectures for simple CRUD applications.

- **The Golden Rule:**

AI Proposes Trade-offs; Lead Architects Make Decisions!

Human Architects must own final architectural decisions and long-term system maintainability.

Human-in-the-Loop: Risks & Blind Spots



Risk 1: Architectural Blind Spots

AI lacks context on team tech-stack familiarity, cloud vendor costs, and organizational culture.



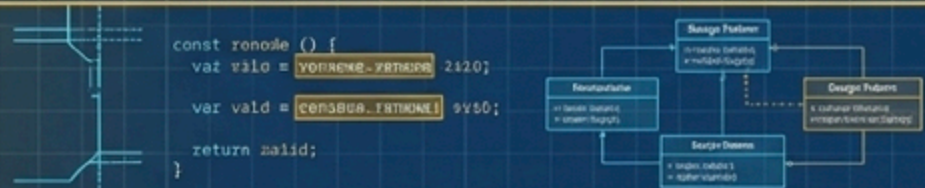
Risk 2: Premature Microservices

The danger of AI recommending complex distributed architectures for simple CRUD applications.



Risk 3: Superficial Refactoring

AI renaming variables or over-engineering with GoF patterns without fixing fundamental architectural flaws.



Golden Rule Callout: AI suggests. The Architect decides.
AI evaluates code, humans evaluate context.

Concept Check Question 2

Which architectural pattern separates user presentation/UI from core data management and user input event handling?

- **A.** Pipe and Filter Pattern
- **B.** Model-View-Controller (MVC) Pattern
- **C.** Repository Pattern
- **D.** Client-Server Pattern



Concept Check Question 2: Answer

Which architectural pattern separates user presentation/UI from core data management and user input event handling?

- **Correct Answer: B**
- **Explanation:** The MVC pattern decouples system data (Model), user UI (View), and input handling (Controller).



Recap: Fill-in-the-blank Quiz

Test your understanding of the core concepts in this chapter:

1. ____ architecture structures a system into stacked abstract machines where each layer provides services to the layer above.
2. In the ____ pattern, all sub-systems share and exchange data through a central database.
3. Kruchten's ____ View Model presents Logical, Process, Development, and Physical perspectives joined by Scenarios.
4. ____ architecture decomposes a monolithic application into small, independently deployable services with decentralized data.



Recap: Answers & Summary

Here are the completed concepts:

1. **Layered** architecture structures a system into stacked abstract machines where each layer provides services to the layer above.
2. In the **Repository** pattern, all sub-systems share and exchange data through a central database.
3. Kruchten's **4+1** View Model presents Logical, Process, Development, and Physical perspectives joined by Scenarios.
4. **Microservices** architecture decomposes a monolithic application into small, independently deployable services with decentralized data.



References

- **Sommerville Software Engineering Book (Chapter 6)**
 - [Kruchten 4+1 View Model Paper](#)
 - [Martin Fowler Microservices Architecture Guide](#)